

Yenepoya (Deemed to be University).
Project – High Level Design
on
AI-Based Phishing Detection for Retail

Student Name:

Aishah Syed

Industry Mentor: Mr. Shashank Korram

Guided by: Mr. Vikyath

Table of Contents

1. Introduction	3
1.1 Scope of the document	3
1.2 Intended Audience	3
1.3 System overview	3
2. System Design	4
2.1 Application Design	4
2.2 Process Flow	4
2.3 Information Flow	5
2.4 Components Design	6
2.5 Key Design Considerations	7
2.6 API Catalogue	7
3. Data Design	8
3.1 Data Model	8
3.2 Data Access Mechanism	8
3.3 Data Retention Policies	8
3.4 Data Migration	8
4. Interfaces	9
5. State and Session Management	10
6. Caching	11
7. Non-Functional Requirements	12
7.1 Security Aspects	12
7.2 Performance Aspects	12
8. References	13

High-Level Design (HLD): AI-Based Phishing Detection for Retail

1. INTRODUCTION

1.1. SCOPE OF THE DOCUMENT

The objective is to provide a scalable, high-level architecture for an AI-driven Phishing Detection System tailored for retail environments. The system transitions from a local script/notebook to a structured pipeline capable of handling real-time email/SMS ingestion, classification via a trained Naive Bayes model, and automated reporting.

1.2. INTENDED AUDIENCE

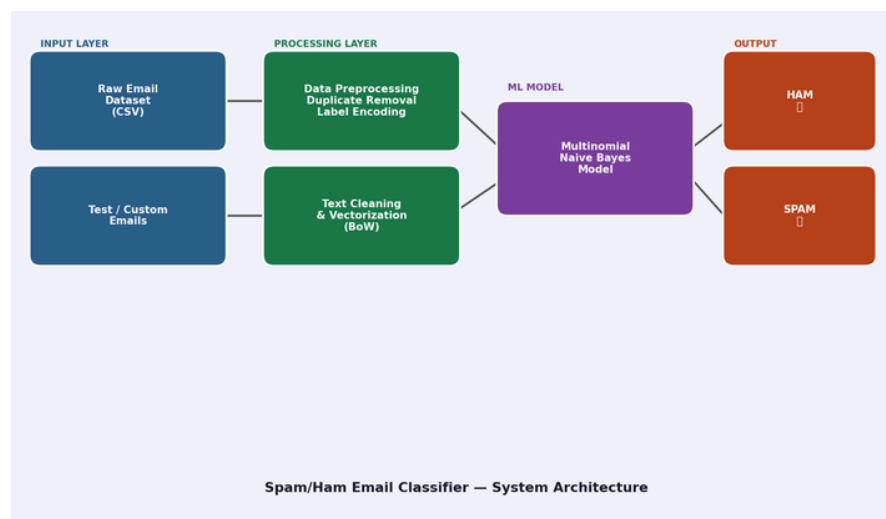
Developers, Data Scientists, and Security Analysts responsible for maintaining or extending the phishing detection model.

1.3. SYSTEM OVERVIEW

The system ingests a labelled SMS/email dataset (CSV format), preprocesses and deduplicates text records, converts raw text into numerical feature vectors using the Bag-of-Words (BoW) model, and trains a Multinomial Naive Bayes classifier. The trained model is capable of classifying unseen messages as either HAM (legitimate) or SPAM with associated confidence probabilities.

Key outcomes achieved on the benchmark dataset:

- Overall Accuracy: 98.16%
- Ham Precision / Recall / F1: 0.99 / 0.99 / 0.99
- Spam Precision / Recall / F1: 0.94 / 0.92 / 0.93



2.SYSTEM DESIGN

2.1. APPLICATION DESIGN

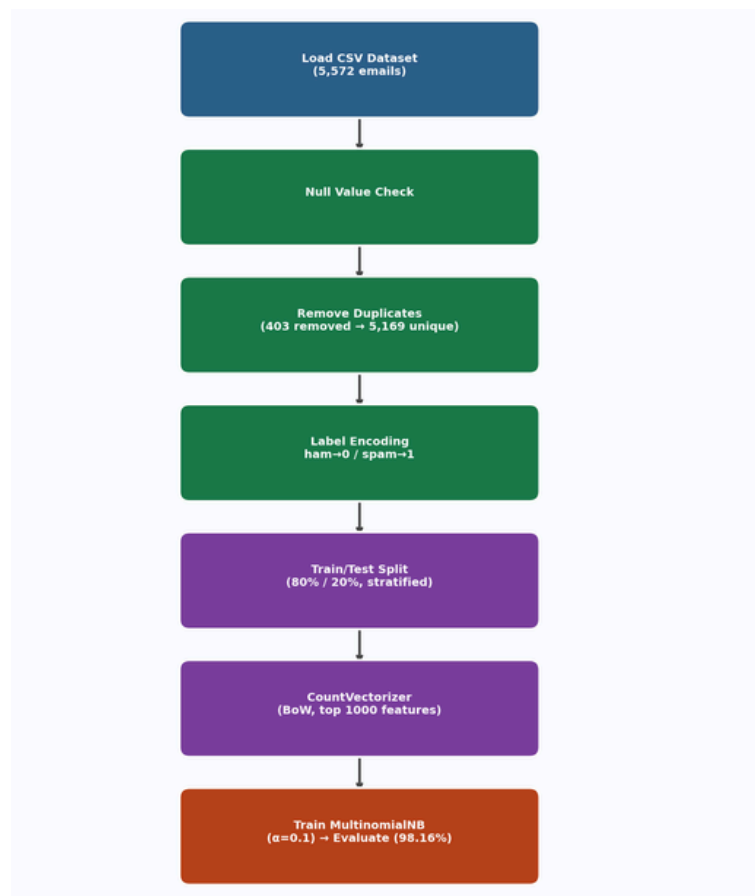
The application follows a linear, batch-processing pipeline pattern. There is no persistent web service or API layer; all components execute sequentially within a single Python script. The design prioritises simplicity, reproducibility (via fixed random_state), and interpretability of the Naive Bayes probabilistic model.

Design principles applied:

- Separation of concerns: each processing stage is isolated (loading, preprocessing, vectorisation, training, evaluation)
- Stratified splitting to preserve class distribution across train/test partitions
- Smoothing via $\alpha=0.1$ in MultinomialNB to handle unseen vocabulary without zero probability collapse
- Confidence-aware prediction using `predict_proba()` rather than binary labels alone

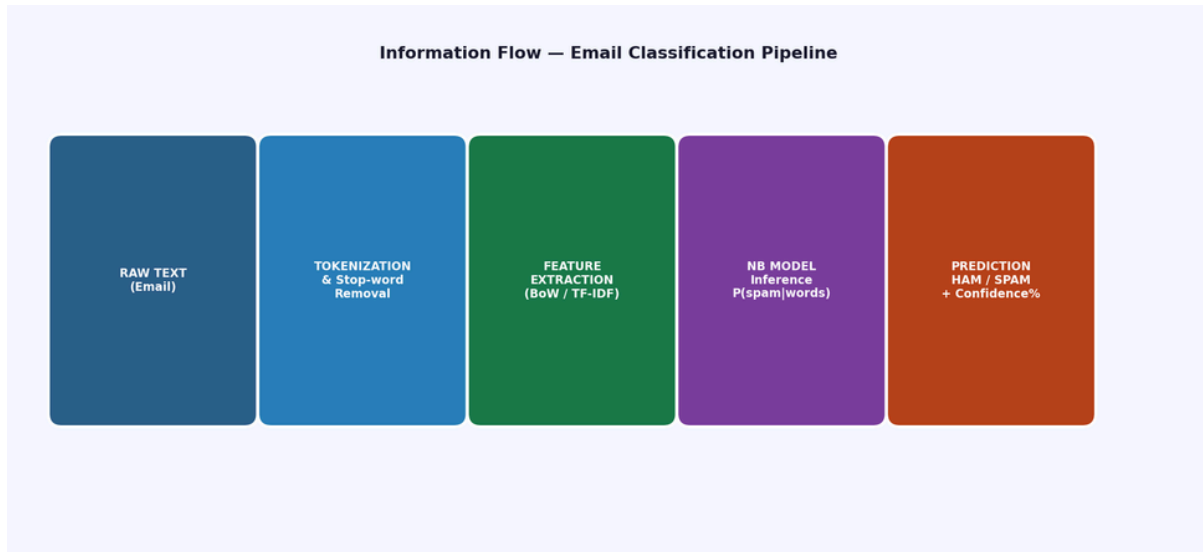
2.2. PROCESS FLOW

The end-to-end process follows seven sequential phases. Data enters as a raw TSV-formatted CSV and exits as a labelled prediction with confidence percentage. The flow is strictly unidirectional with no feedback loops in the base design.



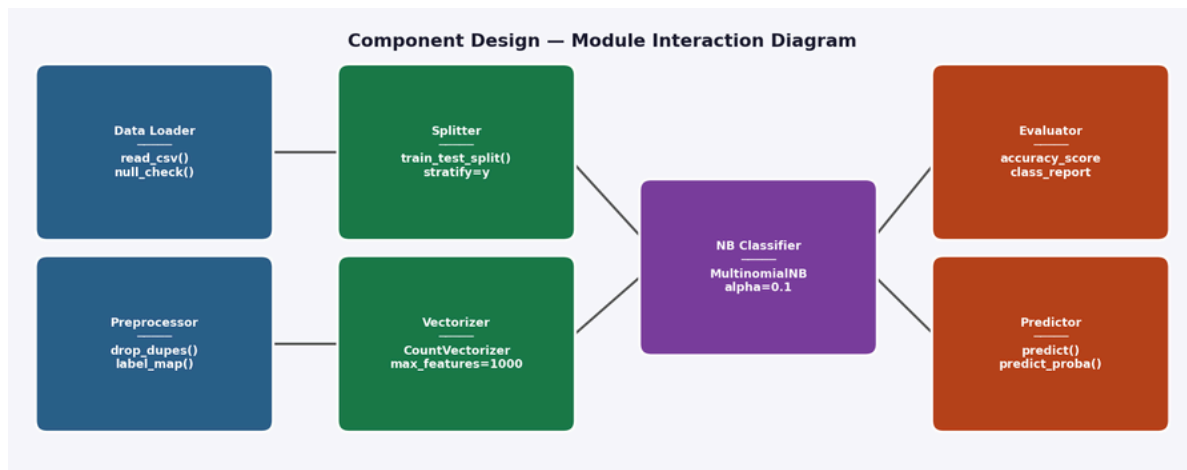
2.3. INFORMATION FLOW

Information moves through five transformation stages. Raw text tokens are progressively abstracted into numerical feature vectors that the probabilistic model can consume. The classifier outputs a posterior probability distribution over the two classes (ham/spam), from which the final label and confidence score are derived.



2.4. COMPONENT DESIGN

The system is composed of six loosely coupled functional modules. Each module encapsulates a specific responsibility and communicates with adjacent modules via pandas DataFrames or scikit-learn compatible arrays.



Component	scikit-learn / Pandas API	Responsibility
Data Loader	pd.read_csv(sep="t")	Ingest raw TSV dataset; expose label and text columns
Preprocessor	df.drop_duplicates()	Remove 403 duplicate text records; map string labels to integers
Splitter	train_test_split(stratify=y)	Partition 80/20 preserving spam/ham ratio in both splits
Vectorizer	CountVectorizer(max_features=1000)	Build BoW vocabulary on train set; transform train & test
Classifier	MultinomialNB(alpha=0.1)	Fit conditional word probabilities; generate class posteriors
Evaluator	classification_report()	Compute accuracy, precision, recall, F1 per class; render confusion matrix

2.5. KEY DESIGN CONSIDERATIONS

- Class Imbalance: The dataset is naturally imbalanced (ham ~87%, spam ~13%). Stratified splitting and class-weighted evaluation metrics mitigate biased accuracy reporting.
- Vocabulary Capping: max_features=1000 limits the feature space, preventing overfitting on rare tokens while keeping inference fast.
- Laplace Smoothing: alpha=0.1 (sub-Laplace) applies mild regularisation; full Laplace (alpha=1.0) was found to reduce spam recall.
- Duplicate Removal: 403 duplicate text messages were identified and removed before splitting to prevent data leakage between train and test sets.
- Stop-word Removal: Common English stop words (e.g., "the", "is") are excluded by CountVectorizer to reduce noise and improve discrimination.

2.6. API CATALOGUE

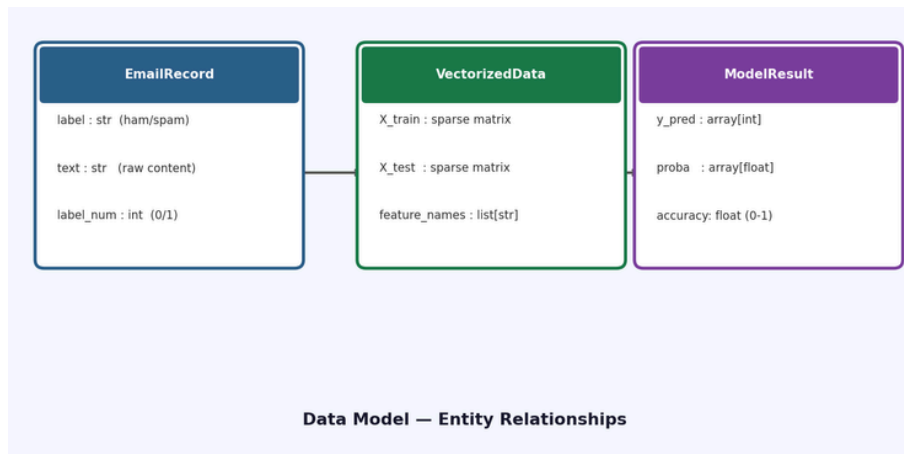
The system exposes no external API endpoints. The following internal function signatures define the public interface of each module:

```
load_data(path: str) -> pd.DataFrame
preprocess(df: pd.DataFrame) -> pd.DataFrame
split(X, y, test_size=0.2, random_state=42) -> Tuple[...]
vectorize(X_train_raw, X_test_raw) -> Tuple[sparse, sparse, CountVectorizer]
train(X_train, y_train, alpha=0.1) -> MultinomialNB
evaluate(model, X_test, y_test) -> dict
predict(model, vectorizer, texts: List[str]) -> List[Tuple[str, float]]
```

3. DATA DESIGN

3.1. DATA MODEL

The primary data artefact is a tab-separated values file (spamhamdata.csv) with no header row. After loading, two derived columns are added: label_num (integer encoding) and — postvectorisation — each unique vocabulary token becomes a feature column in the sparse matrix representation.



3.2. DATA ACCESS MECHANISM

Data is accessed via pandas read_csv with explicit separator (sep="\t") and column names injected programmatically (names=["label","text"]). No database or ORM layer is used. All intermediate transformations operate in-memory as pandas DataFrames or scipy sparse matrices. The CountVectorizer is fit exclusively on training data and serialised implicitly within the model pipeline to prevent test-set leakage.

3.3. DATA RETENTION POLICIES

- Raw dataset: retained on disk at data/spamhamdata.csv; no PII is present
- Trained model: not explicitly persisted in the base design; can be serialised via joblib.dump() for production deployment
- Confusion matrix: saved as confusion_matrix.png for reporting purposes
- Intermediate DataFrames: held in RAM only; released after pipeline completion

3.4. DATA MIGRATION

The system operates on a static dataset with no migration requirement in the current version. For production deployments the following migration considerations apply:

1. New email corpora should be appended with validated ham/spam labels before retraining
2. The CountVectorizer vocabulary should be rebuilt from scratch on each full re-train to capture vocabulary drift
3. Historical confusion matrices should be versioned alongside model snapshots for drift monitoring

4. INTERFACES

The system operates as a standalone command-line script with no graphical or web-based interface in its current form. All inputs and outputs are handled via the file system and standard output (stdout).

Interface	Type	Format	Description
Dataset Input	File (read)	TSV / CSV	Raw labelled email corpus at data/spamhamdata.csv
Prediction Input	In-memory list	Python List[str]	Hardcoded custom email strings for manual testing
Console Output	Stdout	Plain text	Null check, duplicate counts, accuracy, classification report, per-email predictions
Diagram Output	File (write)	PNG	Confusion matrix saved as confusion_matrix.png

5.STATE AND SESSION MANAGEMENT

The current system is stateless between script executions. All state is ephemeral and held exclusively in Python process memory. The following state variables exist during a single execution session:

- df — the cleaned, deduplicated DataFrame (5,169 rows)
- vectorizer — CountVectorizer instance fit on X_train_raw; must be reused for all inference calls
- model — trained MultinomialNB object storing learned log-probabilities
- X_train, X_test — scipy sparse CSR matrices representing BoW feature vectors
- y_train, y_test — integer label arrays (0=ham, 1=spam)

For a production deployment, the vectorizer and model must be jointly serialised (e.g., as a sklearn Pipeline object via joblib) so that prediction-time tokenisation is consistent with trainingtime tokenisation.

6.CACHING

No explicit caching layer is implemented in the base design. The following caching opportunities exist for production scale-up:

- Vectorizer vocabulary cache: The `CountVectorizer.vocabulary_` dictionary can be persisted to disk to avoid rebuilding it on every inference call
- Sparse matrix cache: Vectorized train/test matrices can be saved as `scipy .npz` files to accelerate repeated experiments without re-processing the CSV
- Model cache: The trained `MultinomialNB` can be serialised via `joblib.dump()` and loaded on subsequent runs with `joblib.load()`, eliminating retraining overhead
- Prediction result cache: For batch inference on large corpora, results can be memoised by message hash to avoid re-classifying identical texts

7. NON-FUNCTIONAL REQUIREMENTS

7.1. SECURITY ASPECTS

- Input sanitisation: Email text should be stripped of executable content before classification in any web-facing deployment
- No credential storage: The current system requires no authentication; production deployments should add API key or OAuth2 protection to inference endpoints
- Data privacy: The training corpus contains no personally identifiable information (PII); any future dataset additions must undergo PII screening before inclusion
- Model integrity: The serialised model file should be integrity-checked (SHA-256 hash) before loading to prevent adversarial model substitution

7.2. PERFORMANCE ASPECTS

- Training time: Sub-second on the 5,169-sample dataset on a modern CPU; scales linearly with corpus size
- Inference latency: Single-message classification completes in < 1 ms after vectorisation; suitable for real-time use
- Memory footprint: Sparse BoW matrix for 5,169 samples with 1,000 features occupies approximately 40 MB RAM
- Throughput: Batch prediction via `model.predict()` processes thousands of messages per second using NumPy-backed matrix operations

8.REFERENCES

- Scikit-Learn (Multinomial Naive Bayes):
https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html
- Pandas (Duplicate Handling):
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html
- Matplotlib (Image Persistence/Saving):
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.savefig.html
- Scikit-Learn (Confusion Matrix Display):
<https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ConfusionMatrixDisplay.html>